LOW-LEVEL DESIGN (LLD)

RAG-Based Customer Support Assistant

LangGraph + ChromaDB + Groq LLM + HITL | 2025

1. Module-Level Design

Module	Class/Function	Key Methods
Document Loader	load_and_chunk_pdf()	load PDF/TXT → chunk → return List[Document]
Embedding	HuggingFaceEmbeddings	embed_documents(), embed_query() → List[float]
Vector Store	ChromaDB (langchain-chroma)	from_documents(), similarity_search(query, k)
Graph State	SupportState(TypedDict)	query, retrieved_docs, answer, escalated, messages
Intent Node	intent_node(state, config)	Keyword match → set escalated=True/False
Retriever Node	retriever_node(state, config)	Embed query → ChromaDB search → retrieved_docs
Grader Node	grader_node(state, config)	Slice top-3 chunks → relevant_docs
Generator Node	generator_node(state, config)	Build prompt → ChatGroq.invoke() → answer
HITL Node	hitl_node(state, config)	Return escalation message to user
HITL Manager	HITLManager(SQLite)	create_ticket(), resolve_ticket(), list_tickets()
Graph Builder	build_graph(config_dict)	Compile StateGraph with nodes + conditional edges
Run Query	run_query(graph, query, ...)	Set initial_state → graph.invoke() → return result

2. Key Data Structures

GraphState (TypedDict — shared across all nodes):

```
query: str | retrieved_docs: List[str] | relevant_docs: List[str] | answer: str | confidence: float | escalated: bool | escalation_reason: str | session_id: str | messages: List
```

EscalationTicket (SQLite row):

```
ticket_id: str | query: str | context: str | intent: str | confidence: float | status: str | created_at: str | response: str | resolved_at: str
```

3. LangGraph Workflow & Routing Logic

Node Execution Flow:

Transition	Condition	Next Node
START → intent_node	Always (entry point)	intent_node
intent_node → retriever	escalated == False	retriever_node
intent_node → hitl	escalated == True (keyword match)	hitl_node
retriever → grader	Always	grader_node
grader → generator	Always	generator_node
generator → END	Always	END
hitl → END	Always	END

Routing Function:

```
def route_intent(state): return "hitl" if state["escalated"] else "retriever"
```

Escalation Keywords:

- ESCALATION_KEYWORDS = ["refund", "legal", "lawsuit", "urgent", "fraud", "broken", "damaged", "scam"]
- If ANY keyword found in query.lower() → escalated=True → HITL node
- Confidence < 0.70 OR no chunks retrieved → also triggers HITL

4. Interface & Error Handling

CLI Commands (rag_app.py):

Command	Action
python rag_app.py	Start CLI chat interface
upload	Ingest .txt/.md/.pdf into ChromaDB at runtime
help	Show available commands
quit / exit	Exit the assistant

Error Handling:

Error Scenario	Handling Strategy
No chunks retrieved	escalated=True → HITL node
LLM API failure	Returns [LLM error] message, confidence=0.0
Invalid file upload	File type check before ingestion
Empty query	Skipped with 'continue' in main loop
ChromaDB missing	Re-ingest with python src/ingest.py --text